SÉRIE ORIGINALIUM

ŢŢĻŞŢ~

L'ORIGINE DE L'ART DE L'EXCELLENCE

ŢŢĻŞŢ~ - L'origine de l'art de l'excellence pour la réalisation dans le domaine du logiciel

- PARTIE B -

Léo Michel Bélanger

Fondateur d'ORIGINALIUM

Images et couverture par Soléane Bélanger

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | ŢŢĻŞŢ~

MOT D'OUVERTURE

Une lettre de Léo Michel Bélanger

Cher ami, chère amie,

Tu as franchi la Partie A. Tu as posé les fondations - et ce n'était pas rien. Tu as appris à nommer, à observer, à structurer ta pensée en ingénieur. Tu as commencé à comprendre que le logiciel de qualité ne naît pas du hasard, mais d'une intention maintenue avec rigueur, au quotidien, dans chaque décision, petite ou grande.

Maintenant, c'est différent. La Partie B, c'est le moment où tu vas creuser. Où tu vas descendre sous la surface pour voir ce que la plupart des développeurs ne voient pas - ou font semblant de ne pas voir. Tu vas rencontrer l'ennemi invisible : la dette technique. Cet adversaire silencieux qui s'accumule dans l'ombre, qui ralentit les équipes, qui étouffe l'innovation, et qui transforme une base de code prometteuse en un terrain miné.

Mais tu ne vas pas seulement apprendre à nommer cet ennemi. Tu vas apprendre à le rembourser, à le refactoriser, à reprendre le contrôle. Tu vas découvrir les principes fondateurs du génie logiciel de qualité - SOLID, DRY, KISS, YAGNI - non pas comme des acronymes à mémoriser, mais comme une boussole vivante qui oriente chacune de tes décisions d'architecture et de conception.

Et puis, tu vas apprendre une vérité que peu de formations enseignent vraiment : la qualité ne se contrôle pas en aval, elle se construit en amont. Le mouvement Shift Left, le TDD, le BDD, les révisions de code profondes - ce sont des pratiques de maîtres. Ce sont les outils de ceux qui refusent de réagir quand il est trop tard et qui choisissent d'anticiper avec intelligence.

Ces trois semaines que tu as devant toi ne sont pas de la lecture. Ce sont des exercices de terrain. Chaque question t'oblige à t'arrêter, à réfléchir à ton propre contexte, à ta propre expérience, à ton propre code. C'est là que la vraie maîtrise se

construit - dans ce moment d'honnêteté avec toi-même, stylo en main, face à la page.

Ton chemin vers l'excellence ne commence pas à la fin de ce livre. Il commence ici, maintenant, à chaque exercice que tu prends au sérieux. La Partie B est le cœur de la série ŢŢĻŞŢ~. C'est là que l'artisan devient maître. C'est là que tu commences vraiment.

Alors, prends ton temps. Lis chaque instruction. Écris honnêtement. Et souviens-toi : l'excellence n'est pas une destination - c'est une pratique quotidienne que tu choisis de recommencer chaque jour.

Je suis avec toi dans chaque page de ce livre.

Avec toute ma conviction,

Léo Michel Bélanger

Fondateur d'ORIGINALIUM - ŢŢĻŞŢ~

SEMAINE 4

L'ennemi invisible :la dette technique et la qualité dégradée

ŢŢĻŞŢ~ - Série ORIGINALIUM | Partie B

Introduction & Intention

La dette technique est l'une des réalités les plus mal comprises et les plus sous-estimées du génie logiciel. Elle ne s'annonce pas. Elle s'accumule silencieusement - un raccourci pris ici, une validation ignorée là, une architecture bâclée parce que la livraison pressait. Et un jour, ce qui devait

être un sprint devient un marathon épuisant dans un terrain vaseux.

Cette semaine, tu vas apprendre à voir la dette là où elle se cache. Tu vas comprendre ses mécanismes, mesurer son impact, et surtout, maîtriser les techniques pour la rembourser méthodiquement. Parce qu'ignorer la dette, c'est choisir de la laisser prendre des intérêts - aux dépens de ton équipe, de ton produit, et de ta propre fierté du travail bien fait.

Intention de la semaine : *Développer ta capacité à identifier, quantifier et traiter la dette technique dans un contexte professionnel réel, en appliquant des stratégies de remboursement concrètes et mesurables.*

EXERCICE 4.1

Cartographier la dette dans ta base de code

La dette technique prend plusieurs formes : dette de conception, dette de code, dette de tests, dette documentaire, dette d'infrastructure. Pour chacune de ces catégories, identifie un exemple concret que tu observes ou as observé dans une base de code réelle. Décris brièvement la situation et l'impact qu'elle génère sur le développement.

Catégorie de dette	Exemple concret observé	Impact sur le développement
Dette de conception		
Dette de code		
Dette de tests		
Dette documentaire		
Dette d'infrastructure		

Tes observations complémentaires :

EXERCICE 4.2

Mesurer le coût réel de la dette technique

La dette technique n'est pas seulement un problème qualitatif - elle a un coût financier et humain réel. Choisis une dette technique spécifique que tu connais bien dans ton contexte professionnel. Estime le temps additionnel qu'elle génère chaque semaine (en heures), son impact sur le moral de l'équipe, et le coût approximatif sur une année (en heures-équipe). Montre ton raisonnement.

Description de la dette choisie :

Temps additionnel estimé par semaine (heures) et raisonnement :

Impact sur le moral et la productivité de l'équipe :

Coût annuel estimé (heures-équipe × taux horaire si disponible) :

EXERCICE 4.3

Prioriser le remboursement : la matrice Urgence / Impact

Toutes les dettes ne méritent pas d'être remboursées immédiatement. Certaines sont critiques, d'autres peuvent attendre. Identifie cinq dettes techniques de ton environnement

de travail. Place-les dans la matrice ci-dessous selon leur urgence (risque immédiat) et leur impact (bénéfice si remboursée). Ensuite, détermine l'ordre de traitement recommandé.

Dette technique identifiée	Urgence (1-5)	Impact si remboursée (1-5)	Priorité de traitement

Ton raisonnement sur l'ordre de priorité retenu :

EXERCICE 4.4

Planifier un refactoring stratégique

Le refactoring n'est pas une réécriture complète - c'est une amélioration progressive et disciplinée du code sans modifier son comportement externe. Choisis un module ou une fonction que tu juges problématique. Décris l'état actuel, les problèmes identifiés, et élabore un plan de refactoring en au moins quatre étapes concrètes, avec un critère de succès pour chacune.

Module / fonction choisi(e) et état actuel :

Problèmes identifiés (code dupliqué, couplage fort, nommage ambigu, etc.) :

Étape	Action de refactoring	Critère de succès
1		
2		
3		
4		

EXERCICE 4.5

Analyser l'impact de la dette sur la vélocité de l'équipe

La dette technique ralentit la vélocité des équipes Agile de façon souvent invisible dans les métriques courantes. Sur la base de ton expérience ou d'une situation hypothétique réaliste, décris comment la dette technique affecte : (a) le temps de livraison des nouvelles fonctionnalités, (b) le taux de bugs introduits, (c) le temps d'intégration des nouveaux membres d'équipe, et (d) la satisfaction des développeurs.

(a) Impact sur le temps de livraison des nouvelles fonctionnalités :

(b) Impact sur le taux de bugs introduits :

(c) Impact sur l'intégration des nouveaux membres d'équipe :

(d) Impact sur la satisfaction et la rétention des développeurs :

EXERCICE 4.6

Définir une stratégie de remboursement incrémentale

La règle du Boy Scout en génie logiciel stipule : « Laisse le code dans un meilleur état que celui où tu l'as trouvé. » Conçois une stratégie de remboursement incrémentale pour ton équipe, fondée sur ce principe. Définis : le pourcentage de capacité alloué par sprint, les critères de sélection de la dette à traiter en priorité, les indicateurs de suivi, et le processus de communication avec les parties prenantes non techniques.

Pourcentage de capacité sprint alloué à la dette et justification :

Critères de sélection de la dette à traiter en priorité :

Indicateurs de suivi de la réduction de la dette (métriques concrètes) :

Processus de communication avec les parties prenantes non techniques :

EXERCICE 4.7

Prévenir l'accumulation future : les garde-fous d'équipe

La meilleure dette est celle qu'on ne crée jamais. Conçois un ensemble de garde-fous - pratiques, règles ou automatisations - que ton équipe pourrait adopter pour prévenir l'accumulation non contrôlée de dette technique à l'avenir. Pour chaque garde-fou proposé, indique le type (process, outil, culture) et l'impact attendu sur la qualité à long terme.

Garde-fou proposé	Type (process / outil / culture)	Impact attendu sur la qualité

Le garde-fou que tu juges le plus critique à implanter en premier, et pourquoi :

Questions de réflexion - Semaine 4

1. Quelle est la dette technique que tu as laissé s'accumuler le plus longtemps dans ta carrière, et qu'est-ce que ça t'a coûté ?

2. Comment distingues-tu une décision technique pragmatique (acceptable à court terme) d'une véritable dette irresponsable ?

3. Quel obstacle culturel ou organisationnel t'a empêché de rembourser de la dette dans le passé ? Comment le contournerais-tu aujourd'hui ?

T̰T̰Ṯ̰S̰Ṯ~ - TOTALISATION | SEMAINE 4

Dimension 1 - Additionner et considérer l'ensemble

Prends le temps d'additionner tout ce que tu as produit cette semaine : les dettes cartographiées, les coûts estimés, les refactorings planifiés, les garde-fous définis. Regarde l'ensemble. Qu'est-ce que ces éléments, pris ensemble, te révèlent sur l'état réel de la qualité dans ton environnement de travail ? Écris ta synthèse ici.

Dimension 2 - Rassembler et réaliser totalement : avant, pendant, après, en continu

Rassemble maintenant ta compréhension du cycle complet de la dette : comment elle se forme *avant* (décisions de conception), comment elle croît *pendant* (raccourcis en cours de développement), comment elle se révèle *après* (bugs, lenteur, friction), et comment la gestion de la dette s'inscrit *en continu* dans une pratique d'excellence. Qu'est-ce que tu t'engages à changer dans ta façon de travailler ?

> *La dette technique n'est pas un accident - c'est un choix. Et comme tout choix, elle porte ses conséquences. La différence entre un artisan et un technicien, c'est que l'artisan choisit consciemment quand s'endetter, et ne laisse jamais la dette gouverner à sa place.*
>
> **- Léo Michel Bélanger, ORIGINALIUM ŢŢĻŞŢ~**

SEMAINE 5

Les racines de l'excellence :principes fondateurs du génie logiciel de qualité

TTLST~ - Série ORIGINALIUM | Partie B

Introduction & Intention

SOLID. DRY. KISS. YAGNI. Ces acronymes circulent dans chaque conversation d'architecture, dans chaque revue de code, dans chaque entretien technique. Mais les connaître et les *vivre* sont deux choses radicalement différentes. La plupart des développeurs peuvent te les réciter. Peu peuvent te montrer comment ils ont transformé une décision difficile grâce à eux.

Cette semaine, tu vas faire exactement cela. Ces principes ne sont pas des règles rigides à appliquer mécaniquement - ce sont des heuristiques de sagesse, des outils de discernement que les ingénieurs d'expérience ont forgés à travers des milliers d'heures de code livré, de bugs douloureux, et de refactorings salvateurs. Tu vas apprendre à les sentir, à les appliquer, et à les expliquer à tes pairs.

Intention de la semaine : *Intérioriser les principes SOLID, DRY, KISS et YAGNI comme une boussole de décision active, et comprendre leur impact concret sur la testabilité, la maintenabilité et l'évolutivité d'une architecture logicielle.*

EXERCICE 5.1

SOLID en situation réelle : diagnostiquer les violations

Chacun des cinq principes SOLID peut être violé de manière subtile. Pour chaque principe, décris une violation concrète que tu as observée ou vécue dans un projet réel (ou

hypothétique réaliste). Nomme la violation, explique pourquoi elle la constitue, et décris la solution que tu aurais appliquée.

Principe	Violation observée (exemple concret)	Solution appliquée ou recommandée
S - Single Responsibility		
O - Open/Closed		
L - Liskov Substitution		
I - Interface Segregation		
D - Dependency Inversion		

EXERCICE 5.2

DRY - Repérer et éliminer la duplication

Le principe DRY (Don't Repeat Yourself) ne se limite pas au code dupliqué - il s'applique aussi à la logique métier, aux configurations, aux tests, et même à la documentation. Identifie trois formes différentes de duplication dans un projet que tu connais. Pour chaque forme, explique comment tu l'éliminerais et quel pattern ou mécanisme tu utiliserais (abstraction, constante partagée, template, générateur, etc.).

Duplication #1 - Type et description :

Solution et mécanisme utilisé :

Duplication #2 - Type et description :

Solution et mécanisme utilisé :

Duplication #3 - Type et description :

Solution et mécanisme utilisé :

EXERCICE 5.3

KISS - Simplifier sans simplisme

KISS (Keep It Simple, Stupid) ne signifie pas écrire du code naïf ou éviter la sophistication nécessaire. Il signifie ne pas complexifier au-delà de ce que le problème exige. Décris une situation où tu as vu (ou créé) une solution sur-ingéniérisée, inutilement complexe. Analyse pourquoi cette complexité a été introduite (pression de performance, ego technique, sur-anticipation), et comment une solution plus simple aurait été préférable.

Description de la solution sur-ingéniérisée :

Raison(s) pour laquelle cette complexité a été introduite :

Description de la solution plus simple qui aurait été préférable :

Ce que tu ferais différemment aujourd'hui :

EXERCICE 5.4

YAGNI - La discipline de ne pas anticiper

YAGNI (You Ain't Gonna Need It) est l'un des principes les plus difficiles à respecter pour les développeurs expérimentés, parce qu'il exige de résister à l'envie d'anticiper des besoins futurs qui ne se matérialiseront peut-être jamais. Identifie deux décisions d'architecture ou de développement dans ta carrière où tu as implémenté quelque chose « au cas où » - et où cette anticipation s'est révélée inutile ou coûteuse. Pour chacune, calcule le coût en temps et en complexité ajoutée.

Situation YAGNI #1 - Ce qui a été implémenté « au cas où » :

Coût estimé (temps de développement + maintenance + complexité ajoutée) :

Situation YAGNI #2 - Ce qui a été implémenté « au cas où » :

Coût estimé (temps de développement + maintenance + complexité ajoutée) :

Règle personnelle que tu vas adopter pour mieux appliquer YAGNI :

EXERCICE 5.5

Principes d'architecture : couplage et cohésion

Le couplage faible et la cohésion forte sont deux des piliers architecturaux les plus fondamentaux du génie logiciel de qualité. Analyse un sous-système ou un module de ton environnement de travail. Évalue son niveau de couplage (faible, moyen, fort) et de cohésion (fonctionnelle, communicationnelle, logique, temporelle), et justifie ton évaluation avec des exemples concrets. Propose ensuite une amélioration architecturale ciblée.

Sous-système / module analysé :

Évaluation du couplage (faible / moyen / fort) et justification avec exemples :

Évaluation de la cohésion (type) et justification avec exemples :

Amélioration architecturale ciblée que tu recommanderais :

EXERCICE 5.6

Impact des principes sur la testabilité

La testabilité d'un module est souvent un révélateur direct de la qualité de son architecture. Un module difficile à tester l'est habituellement parce qu'il viole un ou plusieurs des principes fondateurs. Pour chaque principe ci-dessous, explique comment sa violation dégrade spécifiquement la testabilité, et comment son respect la rétablit.

Principe	Comment sa violation dégrade la testabilité	Comment son respect améliore la testabilité
Single Responsibility		
Dependency Inversion		
DRY		
KISS		

EXERCICE 5.7

Créer ta boussole de principes personnelle

À ce stade, tu as une compréhension active des grands principes du génie logiciel. Maintenant, construis ta propre boussole : un ensemble de cinq à sept règles personnelles - formulées dans tes propres mots - que tu t'engages à appliquer dans chaque décision d'architecture ou de développement. Ces règles doivent être concrètes, actionnables, et ancrées dans ton expérience réelle.

#	Ta règle personnelle (dans tes propres mots)	Principe source	Situation où tu l'appliqueras en priorité
1			
2			
3			
4			
5			
6			
7			

Questions de réflexion - Semaine 5

1. Lequel des principes SOLID te coûte le plus de discipline à respecter, et pourquoi selon toi ?

2. Comment expliques-tu l'un de ces principes à un développeur junior ou à une partie prenante non technique ?

3. Y a-t-il des situations dans ton domaine spécifique où ces principes entrent en tension les uns avec les autres ? Lesquelles et comment les arbitres-tu ?

T T L S T ~ - TOTALISATION | SEMAINE 5

Dimension 1 - Additionner et considérer l'ensemble

Additionne les principes que tu as appliqués, les violations que tu as identifiées, les solutions que tu as formulées, et la boussole que tu as construite. Regarde cet ensemble : qu'est-ce que ces principes pris ensemble forment-ils comme vision de l'architecture ? Comment se complètent-ils et se renforcent-ils mutuellement ?

Dimension 2 - Rassembler et réaliser totalement : avant, pendant, après, en continu

Rassemble ta compréhension du cycle de vie des principes : comment les appliques-tu *avant* (lors de la conception), *pendant* (lors du développement et des revues), *après* (lors des refactorings et post-mortems), et *en continu* (comme culture d'équipe permanente) ? Quelle est ta prochaine action concrète pour ancrer ces principes dans ton quotidien professionnel ?

“

Les principes du génie logiciel ne sont pas des lois à subir - ce sont des boussoles à choisir. Celui qui les comprend peut les plier sans les briser. Celui qui les ignore finit toujours par en payer le prix, avec intérêts.

- Léo Michel Bélanger, ORIGINALIUM ŢŢĻṢŢ~

SEMAINE 6

Le cycle vertueux :qualité dès la conception (Shift Left)

ŢŢĻṢŢ~ - Série ORIGINALIUM | Partie B

Introduction & Intention

Il existe deux façons d'aborder la qualité en génie logiciel : la réactiver une fois qu'elle est perdue, ou la construire dès le premier instant. La première est

> épuisante, coûteuse, et génère de la frustration. La seconde est exigeante au départ, mais libératrice sur le long terme.
>
> Le mouvement Shift Left ne consiste pas à ajouter plus de tests à la fin - il consiste à déplacer la réflexion sur la qualité le plus tôt possible dans le cycle de développement. TDD, BDD, révisions de code profondes, définition de qualité en amont : ce sont les pratiques de ceux qui ont choisi d'anticiper plutôt que de réagir. Cette semaine, tu vas maîtriser ces pratiques en profondeur.

Intention de la semaine : *Maîtriser les pratiques Shift Left - TDD, BDD, révisions de code et définition de la qualité en amont - comme des leviers concrets pour construire la qualité dès la conception et réduire le coût des défauts de façon drastique.*

EXERCICE 6.1

Comprendre le Shift Left : le coût des défauts selon leur moment de détection

La règle du 1-10-100 en qualité logicielle stipule qu'un défaut coûte 1 unité à corriger en phase de conception, 10 en phase de développement, et jusqu'à 100 en production. Sur la base de cette règle, calcule et compare le coût de cinq types de défauts selon le moment où ils sont détectés. Utilise des données réelles ou des estimations raisonnées issues de ton expérience.

Type de défaut	Coût si détecté en conception	Coût si détecté en développement	Coût si détecté en production	Facteur multiplicateur observé
Erreur de logique métier				
Violation de sécurité				
Problème de performance				

Type de défaut	Coût si détecté en conception	Coût si détecté en développement	Coût si détecté en production	Facteur multiplicateur observé
Cas limite non géré				
Régression fonctionnelle				

Conclusion : quel est l'argument économique le plus convaincant pour ton organisation en faveur du Shift Left ?

EXERCICE 6.2

TDD - Le cycle Rouge-Vert-Refactor en pratique

Le Test-Driven Development (TDD) suit un cycle strict : 1) Écrire un test qui échoue (Rouge), 2) Écrire le minimum de code pour le faire passer (Vert), 3) Refactoriser sans casser le test (Refactor). Choisis une fonctionnalité simple mais réelle de ton domaine. Décris en détail comment tu appliquerais le cycle TDD complet pour la développer - au moins trois itérations du cycle. Sois précis sur ce que le test valide à chaque étape.

Fonctionnalité choisie et contexte :

Itération	Phase Rouge - Test écrit (que valide-t-il ?)	Phase Verte - Code minimal (quelle logique ?)	Refactor - Amélioration apportée
1			
2			
3			

Bénéfice principal du TDD que tu as constaté ou anticipes pour cette fonctionnalité :

EXERCICE 6.3

BDD - De la spécification au scénario Gherkin

Le Behavior-Driven Development (BDD) transforme les exigences métier en scénarios exécutables, écrits dans un langage que les développeurs, les testeurs et les parties prenantes peuvent tous comprendre. En utilisant la syntaxe Gherkin (Étant donné / Quand / Alors), rédige au moins quatre scénarios BDD complets pour une fonctionnalité de ton domaine métier. Inclus des scénarios de succès, d'erreur et de cas limites.

Fonctionnalité décrite (Feature) :

Scénario 1 - Cas de succès principal :

Étant donné :

Quand :

Alors :

Scénario 2 - Cas d'erreur :

Étant donné :

Quand :

Alors :

Scénario 3 - Cas limite :

Étant donné :

Quand :

Alors :

Scénario 4 - Comportement alternatif :

Étant donné :

Quand :

Alors :

EXERCICE 6.4

Concevoir une révision de code efficace et bienveillante

La révision de code (code review) est l'une des pratiques les plus puissantes pour intégrer la qualité en amont - mais elle est souvent mal pratiquée : trop superficielle, trop lente, ou trop agressive. Conçois un protocole de révision de code pour ton équipe. Définis : les critères de ce qu'une bonne PR doit contenir avant la revue, la liste de contrôle du réviseur, les règles de communication constructive, et les critères d'approbation ou de rejet.

Critères qu'une PR doit satisfaire AVANT la revue (checklist d'auteur) :

Liste de contrôle du réviseur (ce qu'il doit vérifier) :

Règles de communication constructive pendant la revue :

Critères d'approbation et conditions de rejet d'une PR :

EXERCICE 6.5

Définir la qualité en amont : la Definition of Ready et la Definition of Done

Le Shift Left commence bien avant le premier commit - il commence dans la façon dont on définit ce qu'est « prêt à développer » et « terminé ». Crée une Definition of Ready (DoR) et une Definition of Done (DoD) pour ton équipe, adaptées à ton contexte de domaine logiciel. Ces définitions doivent être spécifiques, actionnables et mesurables - pas des généralités.

Definition of Ready (DoR) - Une histoire est prête à développer quand :

Definition of Done (DoD) - Une histoire est terminée quand :

EXERCICE 6.6

Concevoir une stratégie de tests en pyramide

La pyramide de tests (Mike Cohn) propose une répartition optimale des efforts de test : une large base de tests unitaires, une couche intermédiaire de tests d'intégration, et un sommet mince de tests end-to-end. Applique ce modèle à un système logiciel que tu développes ou connais bien. Définis la répartition cible, les outils utilisés à chaque niveau, la couverture visée, et les responsabilités d'équipe.

Niveau de test	Proportion cible (%)	Outils utilisés	Couverture visée	Responsable principal
Tests unitaires				
Tests d'intégration				
Tests de contrat				
Tests end-to-end				

Ce qui manque actuellement dans ta stratégie de tests et comment tu vas le corriger :

EXERCICE 6.7

Implanter le Shift Left : ton plan d'action en 90 jours

La transformation vers une culture Shift Left ne se fait pas du jour au lendemain - elle se construit par étapes, avec des gains rapides qui créent de l'adhésion, suivis de changements plus profonds qui ancrent la pratique. Élabore un plan d'action personnel en 90 jours pour implanter au moins trois pratiques Shift Left dans ton équipe ou ton travail individuel. Sois précis sur les actions, les délais, les obstacles anticipés et les indicateurs de succès.

Période	Pratique Shift Left à implanter	Actions concrètes	Obstacle anticipé et mitigation	Indicateur de succès
Jours 1–30				
Jours 31–60				
Jours 61–90				

Ton engagement personnel envers ce plan (en tes propres mots) :

Questions de réflexion - Semaine 6

1. Qu'est-ce qui te freine le plus dans l'adoption du TDD dans ton contexte actuel, et comment pourrais-tu lever cet obstacle progressivement ?

2. Comment transformerais-tu une culture d'équipe habituée à tester en fin de cycle vers une culture Shift Left, sans créer de résistance ?

3. Quel est le ratio actuel de ta couverture de tests (estimé), et quelle est la cible réaliste que tu vises dans les six prochains mois ? Qu'est-ce qui va changer concrètement ?

Ţ Ţ Ŀ Ş Ţ ~ - TOTALISATION | SEMAINE 6

Dimension 1 - Additionner et considérer l'ensemble

Additionne tous les éléments produits cette semaine : les coûts comparés, les cycles TDD documentés, les scénarios BDD rédigés, le protocole de revue de code, la DoR/DoD, la pyramide de tests, le plan 90 jours. Pris ensemble, que forment-ils ? Quel portrait de ta future pratique de qualité est-il en train d'émerger ? Résume la vision d'ensemble que tu vois se dessiner.

Dimension 2 - Rassembler et réaliser totalement : avant, pendant, après, en continu

Rassemble ta compréhension du Shift Left à travers le cycle complet : comment anticipes-tu la qualité *avant* (DoR, BDD, spécification), comment la construis-tu *pendant* (TDD, revues de code, intégration continue), comment la valides-tu *après* (DoD, métriques de couverture, post-mortems), et comment en fais-tu une culture *en continu* (pratiques d'équipe permanentes) ? Qu'est-ce que ce panorama complet change dans ta façon de concevoir ton rôle d'ingénieur ?

> “
>
> *Réagir à un défaut, c'est subir. Anticiper un défaut, c'est choisir. Le maître artisan du logiciel ne court pas après la qualité - il la construit dans chaque décision, bien avant que le problème n'ait une chance d'exister.*
>
> **- Léo Michel Bélanger, ORIGINALIUM ŢŢĻŞŢ~**

PONT VERS LA PARTIE C

Ce que tu portes maintenant - et ce qui t'attend

Tu arrives au bout de la Partie B. Et je veux que tu prennes un moment pour mesurer ce que tu viens d'accomplir - vraiment.

Tu es entré dans cette partie avec les fondations de la Partie A. Tu en ressors avec quelque chose de fondamentalement différent : une capacité à *voir* ce que la plupart des développeurs ne voient pas. Tu vois la dette là où elle se cache. Tu entends le signal d'alerte quand un principe est violé. Tu sais comment anticiper la qualité au lieu d'en ramasser les dégâts.

Les trois semaines de la Partie B t'ont donné :

- Une **cartographie complète de la dette technique** - ses formes, son coût réel, ses mécanismes de remboursement et les garde-fous pour

l'empêcher de revenir.

- Une **boussole de principes fondateurs** - SOLID, DRY, KISS, YAGNI - formulée dans tes propres mots, ancrée dans ton expérience, prête à guider chaque décision d'architecture à venir.
- Un **arsenal de pratiques Shift Left** - TDD, BDD, révisions de code, Definition of Ready et Definition of Done, pyramide de tests - et un plan d'action en 90 jours pour les ancrer dans ton quotidien professionnel.

Ce sont des outils de maîtres. Pas des théories. Des pratiques que tu as maintenant explorées avec les mains, le stylo, et ton propre contexte.

La **Partie C** va plus loin encore. Elle t'emmène dans la dimension systémique de l'excellence logicielle : comment les équipes de haute performance se construisent, comment les architectures évoluent à grande échelle, comment la culture de la qualité se propage et dure dans le temps. Tu vas découvrir les mécanismes de la Grande Totalisation - le moment où tout ce que tu as appris dans les Parties A et B se fusionne en une vision cohérente et puissante de ce que signifie être un ingénieur d'excellence.

La Partie C est le sommet. Mais souviens-toi : les sommets ne se gravisent qu'avec les fondations solides que tu viens de poser.

Continue. Tu es sur le bon chemin.

Léo Michel Bélanger

ORIGINALIUM - ŢŢĻŞŢ~

Notes personnelles

Notes personnelles

Notes personnelles

ṮṰĻṢṬ~ - FIN DE LA PARTIE B

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | ṮṰĻṢṬ~